📋 QNH Budget System – Complete Documentation & Database Design

1. 🎯 Overview

The Hospital Budget System manages hospital department budgets and validates actual spending against approved expected budgets.

The system does **not replace CareWare**.

CareWare remains responsible for:

- Creating purchase orders
- Approving purchase orders
- Storing PO data

The Budget System is responsible for:

- Creating expected yearly budgets
- Approving department budgets
- Tracking PO usage against approved budget items
- Showing remaining/exceeded balances
- Managing transfers between budget items
- Managing budget history and versions
- Providing reports and audit logs

2. 🔗 CareWare Integration

Integration Type

```
Direct SQL Query from CareWare database
```

CareWare Responsibilities

```
- PO creation
- PO approval
- PO items / PO lines
- PO amount
- PO quantity
- Department related to PO
```

Budget System Responsibilities

```
- Budget creation
- Budget approval
- PO linking after CareWare approval
- Budget balance calculation
- Transfer requests
- Reports
- Audit logs
```

---

3. 👥 Roles

3.1 HOD – Head of Department

The HOD is the main department budget owner.

Can Do

```
- Create department budget
- Enter budget manually
- Import budget from Excel
- Copy budget from previous history
- Submit budget for approval
- Link approved CareWare PO lines to budget items
- Request transfer from one item to another
- Request transfer to a new item
```

3.2 Department User

Department users can work on the budget only if they have permission.

Can Do Depending on Permission

```
- View department budget
- Edit draft budget
- Import Excel budget
- Link PO lines
- Request transfers
```

3.3 Budget Approver

The Budget Approver reviews department budgets and transfer requests.

Can Do

```
- View all or selected department budgets
- Approve budgets
- Reject budgets
- Add general approval/rejection notes
- Add item-specific approval/rejection notes
- Approve transfers
- Reject transfers
- View reports
```

3.4 Admin

The Admin manages setup and access.

Can Do

```
- Add users to Budget System access
- Assign roles
- Give extra permissions
- Manage categories
- Manage types/items
- Review new item requests
- View audit logs
```

---

4. 🔐 Access and Permission Logic

4.1 Existing Users Table

The system already has a main `users` table.

So the Budget System will NOT create a new users table.

Instead:

```
All hospital users exist in the main users table.
Only users added to budget_user_roles can access the Budget System.
```

4.2 Access Rule

```
If user does not exist in budget_user_roles with is_active = 1
→ Access denied
```

Example SQL:

```sql
SELECT 1
FROM budget_user_roles
WHERE user_id = @UserId
AND is_active = 1;
```

4.3 Roles + Extra Permissions

The system uses a hybrid permission model.

```
Role gives default permissions.
User row can override permissions.
```

Permission Logic

```
IF user override permission is NOT NULL
    Use user override
ELSE
    Use role permission
ELSE
    Deny
```

Meaning of Override Values

```
NULL = inherit from role
1 = allow
0 = deny
```

4.4 Example

User has role:

```
DEPARTMENT_USER
```

Role permission:

```
can_view_budget = 1
can_link_po = 0
```

User override:

```
can_link_po = 1
```

Final result:

```
User can view budget from role.
User can link PO from override.
```

---

5. 📊 Budget Main Rules

5.1 Main Rule

```
One ACTIVE APPROVED budget per department per year.
Multiple versions are allowed for history.
```

This means:

```
IT Department can have many budget versions for 2026,
but only one approved active budget for 2026.
```

5.2 Budget Statuses

```
DRAFT
PENDING_APPROVAL
APPROVED
REJECTED
CANCELLED
```

5.3 Edit Rules by Status

Status	Editable?	Notes
DRAFT	Yes	User can edit before submission
PENDING_APPROVAL	No	Waiting for approver decision
APPROVED	No	Locked and used for budget tracking
REJECTED	No	Kept as history/template
CANCELLED	No	Cancelled record kept for audit

---

6. 📥 Budget Creation Methods

A department budget can be created using:

```
1. Manual entry
2. Excel import
3. Copy from previous budget history
```

6.1 Manual Entry

User enters:

```
Category
Type / Item
Quantity
Unit Price
Total Amount
```

Example:

Category	Type	Quantity	Unit Price	Total
IT Equipment	PC	10	5,000	50,000
IT Equipment	Printer	10	1,000	10,000

6.1.1 Budget Item Quantity Distribution

The system allows the user to define how the planned quantity of each budget item will be distributed during the year.

This feature is used for planning and reporting purposes.

It does NOT replace the approved total quantity or total amount stored in `budget_items`.

Storage Principle

```
budget_items = stores the approved budget item total
budget_item_distribution = stores only the planned quantity breakdown by period
```

The distribution table should NOT store calculated amount.

The distribution amount can be calculated when needed:

```
Distribution Amount = Distribution Quantity × budget_items.unit_price
```

Distribution Methods

```
ANNUAL
QUARTERLY
MONTHLY
CUSTOM
```

Option 1: Annual

The full quantity is planned for the entire year without monthly or quarterly breakdown.

Example:

```
Item: PC
Quantity: 50
Unit Price: 5,000
Total Amount: 250,000
Distribution Method: ANNUAL
Distribution Level: YEAR
```

Stored in `budget_items`

id	type	quantity	unit_price	total_amount	distribution_method	distribution_level
101	PC	50	5,000	250,000	ANNUAL	YEAR

Stored in `budget_item_distribution`

```
No rows are required for ANNUAL distribution.
```

Reason:

```
The annual total already exists in budget_items.
```

Option 2: Quarterly Auto Distribution

The system automatically distributes the quantity across four quarters.

Example:

```
Item: PC
Quantity: 50
Unit Price: 5,000
Total Amount: 250,000
Distribution Method: QUARTERLY
Distribution Level: QUARTER
```

The system should NOT distribute countable items like this:

```
Q1 = 12.5 PCs
Q2 = 12.5 PCs
Q3 = 12.5 PCs
Q4 = 12.5 PCs
```

For countable items, the system distributes using whole numbers:

```
Q1 = 13
Q2 = 13
Q3 = 12
Q4 = 12
Total = 50
```

Stored in `budget_items`

id	type	quantity	unit_price	total_amount	distribution_method	distribution_level
101	PC	50	5,000	250,000	QUARTERLY	QUARTER

Stored in `budget_item_distribution`

id	budget_item_id	period_type	period_no	quantity
1	101	QUARTER	1	13
2	101	QUARTER	2	13
3	101	QUARTER	3	12
4	101	QUARTER	4	12

Calculated Amount When Needed

Quarter	Quantity	Unit Price	Calculated Amount
Q1	13	5,000	65,000
Q2	13	5,000	65,000
Q3	12	5,000	60,000
Q4	12	5,000	60,000

Option 3: Monthly Auto Distribution

The system automatically distributes the quantity across twelve months.

Example:

```
Item: PC
Quantity: 50
Unit Price: 5,000
Total Amount: 250,000
Distribution Method: MONTHLY
Distribution Level: MONTH
```

For countable items, the system avoids fractional quantities.

Example monthly distribution:

```
Jan = 5
Feb = 5
Mar = 4
Apr = 4
May = 4
Jun = 4
Jul = 4
Aug = 4
Sep = 4
Oct = 4
Nov = 4
```

```
Dec = 4
Total = 50
```

Stored in `budget_items`

id	type	quantity	unit_price	total_amount	distribution_method	distribution_level
101	PC	50	5,000	250,000	MONTHLY	MONTH

Stored in `budget_item_distribution`

id	budget_item_id	period_type	period_no	quantity
1	101	MONTH	1	5
2	101	MONTH	2	5
3	101	MONTH	3	4
4	101	MONTH	4	4
5	101	MONTH	5	4
6	101	MONTH	6	4
7	101	MONTH	7	4
8	101	MONTH	8	4
9	101	MONTH	9	4
10	101	MONTH	10	4
11	101	MONTH	11	4
12	101	MONTH	12	4

Calculated Amount When Needed

```
Monthly Amount = Month Quantity × budget_items.unit_price
```

Example:

```
January amount = 5 × 5,000 = 25,000
March amount = 4 × 5,000 = 20,000
```

Option 4: Custom Monthly Distribution

The user manually enters specific monthly quantities.

Example:

```
Item: PC
Quantity: 50
Unit Price: 5,000
Total Amount: 250,000
Distribution Method: CUSTOM
Distribution Level: MONTH
```

User enters:

```
January = 10
March = 10
December = 30
Total = 50
```

The system stores only the months that have quantity.

Stored in `budget_items`

id	type	quantity	unit_price	total_amount	distribution_method	distribution_level
101	PC	50	5,000	250,000	CUSTOM	MONTH

Stored in `budget_item_distribution`

id	budget_item_id	period_type	period_no	quantity
1	101	MONTH	1	10
2	101	MONTH	3	10
3	101	MONTH	12	30

Calculated Amount When Needed

```
January amount = 10 × 5,000 = 50,000
March amount = 10 × 5,000 = 50,000
December amount = 30 × 5,000 = 150,000
```

Option 5: Custom Quarterly Distribution

The user manually enters specific quarterly quantities.

Example:

```
Item: PC
Quantity: 50
Unit Price: 5,000
Total Amount: 250,000
Distribution Method: CUSTOM
Distribution Level: QUARTER
```

User enters:

```
Q1 = 20
Q2 = 30
Q3 = 0
Q4 = 0
Total = 50
```

The system may store only periods with quantity greater than zero.

Stored in `budget_items`

id	type	quantity	unit_price	total_amount	distribution_method	distribution_level
101	PC	50	5,000	250,000	CUSTOM	QUARTER

Stored in `budget_item_distribution`

id	budget_item_id	period_type	period_no	quantity
1	101	QUARTER	1	20
2	101	QUARTER	2	30

Calculated Amount When Needed

```
Q1 amount = 20 × 5,000 = 100,000
Q2 amount = 30 × 5,000 = 150,000
```

Validation Rules

Before saving or submitting the budget item, the system must validate:

```
✓ Sum of distributed quantities = budget item total quantity
✓ No negative quantity values
✓ Distribution period type must match the selected method
✓ Custom monthly distribution must use MONTH periods
```

```
✓ Custom quarterly distribution must use QUARTER periods
✓ ANNUAL distribution must not create rows in budget_item_distribution
```

The system does not need to validate distributed amount as stored data because amount is not stored in `budget_item_distribution`.

When needed, distributed amount is calculated using:

```
Distribution Amount = Distribution Quantity × Unit Price
```

System Behavior

```
✓ Annual distribution creates no distribution rows
✓ Quarterly auto distribution creates four QUARTER rows
✓ Monthly auto distribution creates twelve MONTH rows
✓ Custom monthly distribution stores only selected MONTH rows
✓ Custom quarterly distribution stores only selected QUARTER rows
✓ Auto distribution avoids decimals for countable items
✓ Custom distribution allows user control
✓ Distribution totals must match the approved item quantity
✓ Distribution amount is calculated when needed, not stored
```

6.2 Excel Import

User uploads Excel file.

System should:

```
1. Read Excel file
2. Validate columns
3. Validate categories/types
4. Show preview
5. Insert valid rows as budget items
```

Excel import should behave like manual entry.

6.3 Copy From Previous Budget

User can copy a budget from:

```
- Previous rejected budget (same year)
- Previous approved budget (same year)
```

```
- Any old budget from history (any year)
- Previous year approved budget
```

🔁 Copy Behavior Rules

Case A: Same Year Copy (After Rejection)

Example:

```
IT Budget 2026 v1 → REJECTED
User copies it
```

System creates:

```
year = 2026
version_no = 2
status = DRAFT
copied_from_budget_id = old budget id
```

Case B: Copy From Previous Year (NEW IMPORTANT CASE)

Example:

```
IT Budget 2026 v2 → APPROVED
User creates budget for 2027 from it
```

System creates:

```
year = 2027
version_no = 1
status = DRAFT
copied_from_budget_id = 2026 approved budget id
```

🔄 System Behavior

When copying from ANY budget:

```
1. Create new budget record
2. Set status = DRAFT
```

```
3. Set copied_from_budget_id = source budget
4. Generate correct version_no:
   - Same year → version + 1
   - New year → version = 1
5. Copy all budget items
6. Allow user to edit before submitting
```

🗒️ Real DB Example

Old Budget (2026 Approved)

id	department_id	year	version_no	status
2	10	2026	2	APPROVED

New Budget (2027 Created from 2026)

id	department_id	year	version_no	status	copied_from_budget_id
3	10	2027	1	DRAFT	2

📦 Copied Items Example

Old Items (2026)

id	budget_id	type	qty	amount
201	2	PC	10	50,000
202	2	Printer	5	5,000

New Items (2027)

id	budget_id	type	qty	amount	copied_from_item_id
301	3	PC	10	50,000	201
302	3	Printer	5	5,000	202

🖍️ User Actions After Copy

User can:

```
✓ Submit without changes
✓ Edit quantities
✓ Edit prices
```

```
✓ Add new items
✓ Remove items
```

🧠 Final Concept

```
Same Year Copy → New Version
Different Year Copy → New Budget (Version 1)
```

⚠️ Important Rules

```
✓ copied_from_budget_id can reference ANY previous budget
✓ Cross-year copy is allowed
✓ Copy always creates NEW record (never reuse old)
✓ Old budget remains unchanged
```

---

7. 🔁 Budget Versioning

7.1 Why Versioning Exists

Budgets should never be overwritten.

Versioning allows the system to keep:

```
- What user submitted first
- What approver rejected
- What notes approver added
- What user changed later
- Which version became approved
```

7.2 Version Rule

```
Every new attempt creates a new budget record.
Old rejected or approved budgets are never edited.
```

7.3 Example: IT Budget 2026

First Attempt

User creates budget:

Budget ID	Department	Year	Version	Status
1	IT	2026	1	DRAFT

Budget items:

Item ID	Budget ID	Type	Quantity	Unit Price	Total
101	1	PC	10	5,000	50,000
102	1	Printer	10	1,000	10,000

User submits:

Budget ID	Version	Status
1	1	PENDING_APPROVAL

---

8. ❌ Budget Rejection Logic

8.1 What Happens When Approver Rejects

When approver rejects:

```
1. Budget status becomes REJECTED
2. Rejected budget stays in database
3. Rejection notes are saved
4. User cannot edit rejected budget
5. User creates a new version from it
```

8.2 General Rejection Note

A general note is related to the full budget.

Example:

```
Total budget amount is too high. Please reduce IT equipment cost.
```

Stored in budget_notes table with:

```
budget_item_id = NULL
```

8.3 Item-Specific Rejection Note

An item note is related to one budget item.

Example:

```
Printer quantity is too high compared to last year usage.
```

Stored in notes table with:

```
budget_item_id = printer item id
```

8.4 Real DB Example: Reject Budget v1

budget_notes

id	budget_id	budget_item_id	note_type	note	created_by
1	1	NULL	GENERAL_REJECTION	Total budget is too high. Please reduce IT equipment cost.	55
2	1	102	ITEM_REJECTION	Printer quantity is too high compared to last year usage.	55

---

9. 🔁 Create New Version From Rejected Budget

9.1 User Action

User clicks:

```
Create New Version From This Budget
```

9.2 System Action

System creates new budget:

id	department_id	year	version_no	status	copied_from_budget_id
1	10	2026	1	REJECTED	NULL
2	10	2026	2	DRAFT	1

9.3 System Copies Items

Original rejected items:

id	budget_id	Type	Quantity	Total
101	1	PC	10	50,000
102	1	Printer	10	10,000

Copied new items:

id	budget_id	Type	Quantity	Total	copied_from_item_id
201	2	PC	10	50,000	101
202	2	Printer	10	10,000	102

9.4 User Edits New Version

User changes printer quantity:

id	budget_id	Type	Quantity	Unit Price	Total
201	2	PC	10	5,000	50,000
202	2	Printer	5	1,000	5,000

Important:

```
Budget v1 remains unchanged.
Budget v2 is the editable draft.
```

---

10. ✅ Budget Approval Logic

10.1 Approver Approves Budget v2

After user submits v2:

id	year	version_no	status
2	2026	2	PENDING_APPROVAL

Approver approves:

id	year	version_no	status	approved_by
2	2026	2	APPROVED	55

Final history:

id	year	version_no	status
1	2026	1	REJECTED
2	2026	2	APPROVED

10.2 Active Budget Rule

Only this budget is now active for department/year:

```
Department IT
Year 2026
Budget v2
Status APPROVED
```

---

11. 📄 PO Workflow

11.1 Main Flow

```
HOD creates PO in CareWare
↓
Purchasing Manager approves PO in CareWare
↓
User opens Budget System
↓
User selects approved PO
↓
User selects PO line/item
↓
User links PO line to suitable budget item
↓
System updates balance calculation
```

11.2 Important Rule

```
PO linking happens AFTER PO is approved in CareWare.
```

11.3 PO with Multiple Items

One PO can contain multiple CareWare items.

Example:

```
PO-1001
- Dell PC = 20,000
- HP Printer = 5,000
```

User links each PO line separately:

PO Number	PO Line	Budget Item
PO-1001	Dell PC	PC
PO-1001	HP Printer	Printer

11.4 PO Linking Rules

```
- Only approved PO lines can be linked
- PO must belong to same department
- PO date should be inside budget year
- Same PO line cannot be linked twice
- PO line amount must be valid
- One PO can have many lines
- Each PO line is linked separately
```

---

12. 💰 Budget Balance Calculation

12.1 Main Formula

```
Current Balance =
Approved Budget Amount
+ Approved Transfer In
```

```
- Approved Transfer Out
- Linked PO Amount
```

12.2 Quantity Formula

```
Remaining Quantity =
Approved Quantity
+ Transfer In Quantity
- Transfer Out Quantity
- Linked PO Quantity
```

12.3 Balance Calculation Policy

The system **must not persist (store) the calculated balance** of any budget item as a physical column in the database.

Instead, the current balance shall be **derived dynamically at runtime** using transactional data from:

- Approved budget item amounts
- Approved transfer transactions (inbound and outbound)
- Linked Purchase Order (PO) amounts

Implementation Requirement

... All balance calculations must be based ONLY on APPROVED transactions:

- Approved transfers
- Approved budgets
- Approved PO links

---

13. ⚠️ Over Budget Case

The Budget System does not block PO linking if CareWare already approved the PO.

Example:

Item	Approved Budget	Linked PO	Remaining
Printer	10,000	12,000	-2,000

System should:

```
- Allow linking
- Show negative balance
```

```
- Mark item as EXCEEDED
- Include it in exceeded budget reports
```

---

14. 🔁 Transfer Logic

14.1 Transfer Types

```
1. Existing item → Existing item
2. Existing item → New item
```

14.2 Transfer Requires Approval

All transfer requests must be approved by the Budget Approver.

Flow:

```
User requests transfer
↓
Status = PENDING_APPROVAL
↓
Approver approves or rejects
↓
If approved, transfer affects balance
```

14.3 Transfer Input Methods

The system supports two methods for initiating a transfer between budget items:

```
1. Transfer by Amount
2. Transfer by Target Quantity
```

Option 1: Transfer by Amount

The user specifies the monetary value to transfer from the source item to the target item.

Example

```
Transfer 5,000 from PC to Printer
```

Calculation

```
Source quantity deducted = 5,000 / 5,000 = 1 PC
Target quantity added = 5,000 / 2,000 = 2.5 Printers
```

Result

Item	Quantity Change	Amount Change
PC	-1	-5,000
Printer	+2.5	+5,000

Note

```
Fraction quantities are allowed and must not be rounded automatically.
```

Option 2: Transfer by Target Quantity

The user specifies the quantity required for the target item.

Example

```
Transfer enough budget to add 5 Printers from PC
```

Calculation

```
Required transfer amount = 5 × 2,000 = 10,000
Source quantity deducted = 10,000 / 5,000 = 2 PCs
```

Transfer to New Item

If the target item does not exist in the approved budget, the user may request a transfer to a new item.

```
The user must provide:
```

```
- Category
- Type / Item name
- Unit price
- Transfer amount OR target quantity

## Validation Rules
```

Before processing the transfer request, the system must validate:

```
- Source item must have sufficient available balance
- Requested transfer amount must NOT exceed available balance
- Source item must have positive balance
- Transfer amount must be greater than zero
- Target item must exist or be defined
- Transfer request must follow approval workflow
```

Critical Rule

```
If the requested transfer exceeds available balance,
the system MUST reject the request.
```

Validation Enhancement: Maximum Transfer Suggestion

If validation fails, the system shall:

```
1. Reject the request
2. Calculate maximum transferable amount
3. Suggest maximum allowed value to the user
```

Example (Rejected Case)

Item	Available Balance
PC	5,000

Request:

```
Transfer 5 Printers (requires 10,000)
```

Result:

```
❌ Rejected – insufficient balance
✓ Maximum allowed:
   Amount: 5,000
   Quantity: 2.5 Printers
```

Transfer Behavior Summary

```
✓ Transfer updates BOTH amount and quantity
✓ Amount is the source of truth
✓ Quantity is derived from unit price
✓ No transfer allowed if balance is insufficient
```

---

15. 🏭 Categories, Types, and New Item Requests

15.1 Structure

```
Category
  └── Type / Item
```

Example:

```
IT Equipment
  ├── PC
  ├── Printer
  └── Scanner
```

15.2 Category and Type Request (Combined)

If the user cannot find the required type/item in the dropdown, the system supports requesting both **category and type together**.

Request Flow

User submits:

```
Requested Category (optional)
Requested Type / Item (required)
```

Case A: Category Exists

Example:

```
Category: IT Equipment
Type: 3D Printer
```

Flow:

```
User selects existing category
↓
User enters new type/item
↓
Admin reviews request
↓
If approved → type is added under category
```

Case B: Category Does Not Exist

Example:

```
Category: AI Equipment
Type: AI Server
```

Flow:

```
User submits both category and type
↓
Admin reviews request
↓
If approved:
   1. Create new category
   2. Create new type under that category
↓
Item becomes available in dropdown
```

Important Rules

```
✓ Users cannot directly create categories or types
✓ Users must submit request
✓ Admin controls final structure
✓ Prevents duplicates and inconsistent naming
✓ One request can create BOTH category and type
```

---

16. 📊 Reports

Approver/Admin can view reports by:

```
- Department
- Year
- Category
- Type / Item
- Budget status
- Exceeded / not exceeded
- Remaining balance
- Transfer history
- PO usage
```

16.1 Important Report Calculations

```
Approved Budget = Sum approved budget items
Actual Spending = Sum linked PO amounts
Variance = Approved Budget - Actual Spending
```

---

17. 📃 Audit Logs

System must log all important actions.

Examples:

```
CREATE_BUDGET
UPDATE_DRAFT_BUDGET
IMPORT_EXCEL
SUBMIT_BUDGET
APPROVE_BUDGET
REJECT_BUDGET
```

```
ADD_GENERAL_APPROVAL_NOTE
ADD_ITEM_APPROVAL_NOTE
ADD_GENERAL_REJECTION_NOTE
ADD_ITEM_REJECTION_NOTE
COPY_BUDGET_FROM_HISTORY
CREATE_NEW_VERSION
LINK_PO
REQUEST_TRANSFER
APPROVE_TRANSFER
REJECT_TRANSFER
CREATE_ITEM_REQUEST
APPROVE_ITEM_REQUEST
REJECT_ITEM_REQUEST
ASSIGN_USER_ROLE
UPDATE_USER_PERMISSION
```

---

18. 🗄 Database Design

18.1 Budget Roles

```
budget_roles (
 id INT PRIMARY KEY,
 name VARCHAR(50) UNIQUE NOT NULL,
 description VARCHAR(200)
)
```

Example roles:

```
HOD
DEPARTMENT_USER
BUDGET_APPROVER
ADMIN
```

18.2 Budget Role Permissions

```
budget_role_permissions (
 id BIGINT PRIMARY KEY,
 role_id INT NOT NULL,

 can_view_budget BIT DEFAULT 0,
 can_edit_budget BIT DEFAULT 0,
 can_link_po BIT DEFAULT 0,
```

```
 can_request_transfer BIT DEFAULT 0,
 can_approve_budget BIT DEFAULT 0,
 can_approve_transfer BIT DEFAULT 0,
 can_manage_users BIT DEFAULT 0,
 can_manage_categories BIT DEFAULT 0,
 can_view_reports BIT DEFAULT 0,

 FOREIGN KEY (role_id) REFERENCES budget_roles(id)
)
```

18.3 Budget User Roles

```
budget_user_roles (
 id BIGINT PRIMARY KEY,
 user_id INT NOT NULL,
 department_id INT NULL,
 role_id INT NULL,

 can_view_budget BIT NULL,
 can_edit_budget BIT NULL,
 can_link_po BIT NULL,
 can_request_transfer BIT NULL,
 can_approve_budget BIT NULL,
 can_approve_transfer BIT NULL,
 can_manage_users BIT NULL,
 can_manage_categories BIT NULL,
 can_view_reports BIT NULL,

 is_active BIT DEFAULT 1,
 created_by INT NULL,
 created_at DATETIME DEFAULT GETDATE(),
 updated_at DATETIME NULL,

 FOREIGN KEY (role_id) REFERENCES budget_roles(id)
)
```

Constraint:

```
ALTER TABLE budget_user_roles
ADD CONSTRAINT UQ_budget_user_roles
UNIQUE (user_id, department_id, role_id);
```

18.4 Budgets

```
budgets (
 id BIGINT PRIMARY KEY,
 department_id INT NOT NULL,
 year INT NOT NULL,
 version_no INT NOT NULL,
 status VARCHAR(50) NOT NULL,

 copied_from_budget_id BIGINT NULL,

 submitted_by INT NULL,
 submitted_at DATETIME NULL,

 approved_by INT NULL,
 approved_at DATETIME NULL,

 rejected_by INT NULL,
 rejected_at DATETIME NULL,

 created_by INT NOT NULL,
 created_at DATETIME DEFAULT GETDATE(),
 updated_at DATETIME NULL,

 is_active BIT DEFAULT 1,

 FOREIGN KEY (copied_from_budget_id) REFERENCES budgets(id),

 UNIQUE(department_id, year, version_no)
)


ALTER TABLE budgets
ADD CONSTRAINT CK_budgets_status
CHECK (status IN ('DRAFT','PENDING_APPROVAL','APPROVED','REJECTED','CANCELLED'));
```

Only one active approved budget per department/year:

```
CREATE UNIQUE INDEX UQ_approved_budget_per_department_year
ON budgets(department_id, year)
WHERE status = 'APPROVED' AND is_active = 1;
```

18.5 Budget Items

```
budget_items (
 id BIGINT PRIMARY KEY,
 budget_id BIGINT NOT NULL,
 category_id INT NOT NULL,
 type_id INT NOT NULL,
```

```
 quantity DECIMAL(18,2),
 unit_price DECIMAL(18,2),
 total_amount DECIMAL(18,2),

 copied_from_item_id BIGINT NULL,

 distribution_method VARCHAR(50) NULL,
 distribution_level VARCHAR(50) NULL,

 created_at DATETIME DEFAULT GETDATE(),
 updated_at DATETIME NULL,
 is_active BIT DEFAULT 1,

 FOREIGN KEY (budget_id) REFERENCES budgets(id),
 FOREIGN KEY (copied_from_item_id) REFERENCES budget_items(id)
)
```

18.6 Budget Item Distribution

```
budget_item_distribution (
 id BIGINT PRIMARY KEY,
 budget_item_id BIGINT NOT NULL,

 period_type VARCHAR(50) NOT NULL, -- MONTH or QUARTER
 period_no INT NOT NULL,

 quantity DECIMAL(18,2) NOT NULL,

 created_at DATETIME DEFAULT GETDATE(),

 FOREIGN KEY (budget_item_id) REFERENCES budget_items(id)
)


ALTER TABLE budget_item_distribution
ADD CONSTRAINT CK_period_valid
CHECK (
    (period_type = 'MONTH' AND period_no BETWEEN 1 AND 12)
    OR
    (period_type = 'QUARTER' AND period_no BETWEEN 1 AND 4)
);

CHECK (
 (distribution_method = 'MONTHLY' AND distribution_level = 'MONTH')
 OR
 (distribution_method = 'QUARTERLY' AND distribution_level = 'QUARTER')
 OR
 (distribution_method = 'ANNUAL' AND distribution_level = 'YEAR')
 OR
```

```
 (distribution_method = 'CUSTOM' AND distribution_level IN ('MONTH','QUARTER'))
)
```

18.7 Budget Notes

```
budget_notes (
 id BIGINT PRIMARY KEY,
 budget_id BIGINT NOT NULL,
 budget_item_id BIGINT NULL,
 note_type VARCHAR(50) NOT NULL,
 note NVARCHAR(MAX) NOT NULL,
 created_by INT NOT NULL,
 created_at DATETIME DEFAULT GETDATE(),

 FOREIGN KEY (budget_id) REFERENCES budgets(id),
 FOREIGN KEY (budget_item_id) REFERENCES budget_items(id)
)
```

Note Types

The system supports notes added by the approver during approval or rejection.

```
GENERAL_REJECTION
ITEM_REJECTION
GENERAL_APPROVAL
ITEM_APPROVAL
```

Note Scope

```
GENERAL → applies to the whole budget (budget_item_id = NULL)
ITEM → applies to a specific item (budget_item_id = item id)
```

Usage Rules

```
✓ Rejection notes are REQUIRED when rejecting a budget
✓ Approval notes are OPTIONAL when approving a budget
✓ Notes are added ONLY during approve/reject actions
✓ Notes are NOT used for review or intermediate workflow
```

Examples

General Rejection

```
"Total budget is too high. Please reduce IT equipment cost."
```

Item Rejection

```
"Printer quantity is too high compared to last year usage."
```

General Approval

```
"Budget approved. Please monitor expenses during the year."
```

Item Approval

```
"Approved, but compare vendor prices before purchase."
```

Design Principle

```
budget_notes is the official communication channel between:
- Budget Approver
- HOD / Department
```

All decisions must be documented through notes for audit and tracking.

18.8 Categories

```
budget_categories (
 id INT PRIMARY KEY,
 name VARCHAR(200) NOT NULL,
 is_active BIT DEFAULT 1,
 created_at DATETIME DEFAULT GETDATE()
)
```

18.9 Types / Items

```
budget_types (
 id INT PRIMARY KEY,
 category_id INT NOT NULL,
 name VARCHAR(200) NOT NULL,
 is_active BIT DEFAULT 1,
 created_at DATETIME DEFAULT GETDATE(),

 FOREIGN KEY (category_id) REFERENCES budget_categories(id)
)
```

18.10 Item Requests

```
budget_item_requests (
 id BIGINT PRIMARY KEY,

 requested_category_name VARCHAR(200) NULL,
 requested_type_name VARCHAR(200) NOT NULL,

 existing_category_id INT NULL,

 requested_by INT NOT NULL,
 status VARCHAR(50) NOT NULL, -- PENDING, APPROVED, REJECTED

 admin_note NVARCHAR(MAX) NULL,
 reviewed_by INT NULL,
 reviewed_at DATETIME NULL,

 created_at DATETIME DEFAULT GETDATE(),

 FOREIGN KEY (existing_category_id) REFERENCES budget_categories(id)
)

ALTER TABLE budget_item_requests
ADD CONSTRAINT CK_item_requests_status
CHECK (status IN ('PENDING','APPROVED','REJECTED'));
```

18.11 PO Links

```
budget_po_links (
 id BIGINT PRIMARY KEY,
 budget_item_id BIGINT NOT NULL,

 po_number VARCHAR(100) NOT NULL,
 po_line_id VARCHAR(100) NOT NULL,
 po_line_description NVARCHAR(500) NULL,
```

```
 amount DECIMAL(18,2) NOT NULL,
 quantity DECIMAL(18,2) NULL,

 linked_by INT NOT NULL,
 linked_at DATETIME DEFAULT GETDATE(),

 is_active BIT DEFAULT 1,

 FOREIGN KEY (budget_item_id) REFERENCES budget_items(id),

 UNIQUE(po_line_id)
)
```

18.12 Transfers

```
budget_transfers (
 id BIGINT PRIMARY KEY,
 from_item_id BIGINT NOT NULL,
 to_item_id BIGINT NULL,

 to_category_id INT NULL,
 to_type_id INT NULL,

 amount DECIMAL(18,2) NOT NULL,

 source_quantity DECIMAL(18,2) NULL,
 target_quantity DECIMAL(18,2) NULL,

 transfer_input_type VARCHAR(50) NOT NULL,

 status VARCHAR(50) NOT NULL,
 reason NVARCHAR(MAX) NULL,

 requested_by INT NOT NULL,
 requested_at DATETIME DEFAULT GETDATE(),

 approved_by INT NULL,
 approved_at DATETIME NULL,

 rejected_by INT NULL,
 rejected_at DATETIME NULL,
 rejection_note NVARCHAR(MAX) NULL,

 FOREIGN KEY (from_item_id) REFERENCES budget_items(id),
 FOREIGN KEY (to_item_id) REFERENCES budget_items(id)
)


ALTER TABLE budget_transfers
ADD CONSTRAINT CK_transfer_target
```

```
CHECK (
(to_item_id IS NOT NULL AND to_category_id IS NULL AND to_type_id IS NULL)
OR
(to_item_id IS NULL AND to_category_id IS NOT NULL AND to_type_id IS NOT NULL)
);

ALTER TABLE budget_transfers
ADD CONSTRAINT CK_transfers_status
CHECK (status IN ('PENDING_APPROVAL','APPROVED','REJECTED'));
```

18.13 Audit Logs

```
budget_audit_logs (
 id BIGINT PRIMARY KEY,
 user_id INT NULL,
 action VARCHAR(200) NOT NULL,
 entity VARCHAR(200) NOT NULL,
 entity_id BIGINT NULL,
 old_value NVARCHAR(MAX) NULL,
 new_value NVARCHAR(MAX) NULL,
 ip_address VARCHAR(100) NULL,
 created_at DATETIME DEFAULT GETDATE()
)
```

---

19. 🧠 Complete Real Scenario

Step 1: IT Department Creates Budget v1

Item	Quantity	Unit Price	Total
PC	10	5,000	50,000
Printer	10	1,000	10,000

Status:

```
DRAFT
```

Step 2: HOD Submits Budget

Status becomes:

```
PENDING_APPROVAL
```

Step 3: Approver Rejects

General note:

```
Total amount is high. Reduce IT equipment cost.
```

Item note:

```
Printer quantity is too high compared to last year.
```

Status:

```
REJECTED
```

Step 4: User Creates Version 2 From Version 1

System copies all items.

Status:

```
DRAFT
```

User edits:

```
Printer quantity from 10 to 5
```

Step 5: User Submits Version 2

Status:

```
PENDING_APPROVAL
```

Step 6: Approver Approves Version 2

Status:

```
APPROVED
```

Now this is the active budget.

Step 7: HOD Creates PO in CareWare

CareWare PO:

```
PO-1001
PC = 20,000
Printer = 7,000
```

Purchasing Manager approves PO in CareWare.

Step 8: User Links PO Lines

Budget System links:

PO Line	Budget Item
PC line	PC
Printer line	Printer

Step 9: Balance Calculation

Approved budget:

Item	Approved Amount
PC	50,000
Printer	5,000

Linked PO:

Item	PO Amount
PC	20,000
Printer	7,000

Remaining:

Item	Remaining
PC	30,000
Printer	-2,000

Printer is marked:

```
EXCEEDED
```

Step 10: Transfer Request

User wants to transfer:

```
5,000 from PC to Printer
```

System checks PC balance:

```
PC balance = 30,000
```

Allowed.

Transfer status:

```
PENDING_APPROVAL
```

Approver approves.

New balance:

Item	Remaining
PC	25,000
Printer	3,000

---

20. 🔒 Final Rules

```
✓ Existing users table remains unchanged
✓ Only users in budget_user_roles can access Budget System
✓ One active approved budget per department/year
```

```
✓ Multiple budget versions allowed
✓ Approved budgets cannot be edited
✓ Rejected budgets cannot be edited
✓ New version must be created after rejection
✓ Budget can be copied from history
✓ General notes are allowed
✓ Item-specific notes are allowed
✓ PO is created and approved in CareWare
✓ PO is linked after CareWare approval
✓ PO line can be linked once only
✓ PO over budget is allowed but marked exceeded
✓ Transfers require approval
✓ Cannot transfer if requested amount exceeds available balance
✓ Cannot transfer from negative/zero balance item
✓ Transfer calculations must use ONLY APPROVED transfers (not pending/rejected)
✓ Admin manages users, categories, types, and item requests
✓ All actions must be audited
```

✅ END

Year: 2026

| Item | Method | Total | Q1 | Q2 | Q3 | Q4 | | | | Jan | Feb | Mar | Apr | May | Jun | Jul | Aug | Sep | Oct | Nov | Dec |

| PC | MONTHLY | 50 | 5 | 5 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | | Printer | QUARTERLY | 20 | 5 | - | - | 5 | - | - | 5 | - | - | 5 | - | - | | Scanner | CUSTOM-M | 10 | 2 | 0 | 2 | 0 | 0 | 0 | 0 | 3 | 0 | 0 | 0 | 3 | | Software | ANNUAL | 1 | - | - | - | - | - | - | - | - | - | - | - | | Service | CUSTOM-Q | 100 | 40 | - | - | 30 | - | - | 30 | - | - | 0 | - | - |

```
                YEAR 2026
```

```
      Q1                Q2                Q3                Q4
```

Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec

PC 5 5 4 4 4 4 4 4 4 4 4 4 Printer 5 - - 5 - - 5 - - 5 - - Scanner 2 0 2 0 0 0 0 3 0 0 0 3 Software - - - - - - - - - - - - Service 40 - - 30 - - 30 - - 0 - -